

Model Testing in CI/CD

CompTIA SecAI+: AI Security Certification Complete Exam Prep 2026 | Section 21: CI/CD and AI Automation

1. Why ML Models Require a Different Testing Discipline

Traditional application code has a deterministic relationship between logic and behaviour: given input X, the code reliably produces output Y. A comprehensive test suite can enumerate all expected input-output relationships and verify them. Machine learning models break this deterministic model in three important ways that fundamentally change how testing must work.

First, model behaviour emerges from training data and learned parameters — not just the code. Two identical model architectures trained on different datasets will produce different outputs for the same input. Second, model outputs are probabilistic confidence distributions, not discrete deterministic values. A fraud detection model might output 94% fraud probability for a given transaction; whether this is correct depends on the ground truth label that may not be available until days later. Third, models degrade over time as real-world data distributions shift away from what they were trained on — a phenomenon called model drift — even though the code itself has not changed. These three properties require an entirely different testing discipline: MLOps.

2. MLOps: Applying CI/CD Discipline to the ML Lifecycle

MLOps (Machine Learning Operations) applies the proven principles of DevOps — automation, versioning, continuous validation, fast feedback loops, and gated deployment — to the machine learning development and deployment lifecycle. The analogy to traditional CI/CD is direct but requires adaptation at every stage:

Traditional CI/CD	MLOps Equivalent	Key Difference
Code version control	Model + data version control	Training data and hyperparameters must be versioned alongside code
Unit and integration tests	Accuracy, bias, and robustness tests	Tests are statistical, not deterministic
Build pipeline	Training pipeline	Building a model requires compute resources and training data, not just compilation
Deployment gate	Model validation gate	Gate checks accuracy, fairness, and latency against thresholds
Rollback	Model rollback	Reverting to the previous model version when the new one underperforms
Runtime monitoring	Drift detection and performance monitoring	Model degradation without code change is a normal failure mode

For the SecAI+ exam, MLOps is the operational framework that turns AI governance policy commitments into automated, measurable enforcement. When a regulation requires that AI systems be fair and accurate, MLOps is the mechanism that verifies this continuously rather than at a single point in time.

3. Regulatory and Governance Context

Model testing in CI/CD is the operational implementation of several significant regulatory requirements. Understanding the regulatory context helps you answer scenario questions that connect technical testing to governance obligations.

- NIST AI RMF 1.0 (2023): The AI Risk Management Framework's four functions — GOVERN, MAP, MEASURE, and MANAGE — require ongoing measurement of AI system behaviour. Model CI/CD pipelines operationalise the MEASURE function through automated accuracy, bias, and robustness testing.
- EU AI Act (2024): Mandates conformity assessments for high-risk AI systems used in hiring, credit scoring, medical diagnosis, and law enforcement. Ongoing technical testing in deployment pipelines is part of meeting these assessment requirements.
- NIST CSF 2.0 GV.RM: Risk management requirements apply to AI systems as part of the broader technology risk framework.
- ISO/IEC 42001:2023: Establishes an AI management system standard requiring documented procedures for AI system testing and monitoring.

The practical implication: organisations deploying AI in regulated contexts cannot rely on one-time validation at initial deployment. Continuous testing through MLOps pipelines is required to maintain compliance as models are retrained and real-world conditions evolve.

4. Accuracy Testing Against Holdout Datasets

The most fundamental model test is accuracy: does the model produce correct predictions on data it has never seen before? This requires careful data management to ensure the evaluation is meaningful.

A holdout test dataset must be reserved before training begins — data that was not used for training and not used for hyperparameter tuning. Using training data or validation data to evaluate final performance inflates accuracy estimates and produces models that perform well in the lab but poorly in production. This data leakage failure mode is common in organisations that rush model development.

Task Type	Primary Accuracy Metrics	CI/CD Gate Logic
Binary classification	Precision, Recall, F1, AUC-ROC	New model >= current production model AND >= absolute floor
Multiclass classification	Per-class precision/recall, macro F1	No significant class-level regression from production model
Regression	RMSE, MAE, R-squared	Error metrics do not exceed production model by defined tolerance
Detection/ranking	Mean Average Precision (mAP)	mAP >= production model at defined confidence thresholds

The gate compares new model performance against the current production model and against absolute minimum thresholds. A new model that performs worse than the existing production model should not

be deployed automatically — even if it passes absolute thresholds — without human review and explicit approval.

5. Real-World Incident: The Fraud Detector Drift

Real-World Pattern — Payment Processor Fraud Model (2020): A payment processor deployed a fraud detection model that initially achieved 97% precision and 89% recall. Over 14 months, the fraud landscape shifted as attackers adopted new techniques the training data had not captured. The model's recall gradually degraded to 71% — meaning nearly 30% of fraud went undetected. The degradation was gradual enough to be invisible without systematic monitoring. Annual model retraining did not keep pace with the rate of fraud technique evolution. Monthly accuracy regression testing with defined threshold alerts would have triggered a retraining cycle before the gap reached business-critical levels. Continuous model testing is the operational discipline that catches this class of failure.

This incident illustrates both model drift (performance degradation without code change) and the inadequacy of periodic rather than continuous testing. MLOps pipeline monitoring would have flagged the recall decline at the first threshold breach and triggered automated retraining or human escalation.

6. Bias and Fairness Testing

AI models can encode and amplify biases present in training data, producing systematically unfair outcomes for specific demographic groups. Bias testing is a first-class concern in model CI/CD — not an optional extra applied only when regulators ask.

Three fairness metrics appear regularly in both industry practice and regulatory requirements:

- **Demographic parity:** Positive prediction rates should be approximately equal across demographic groups. A hiring model with demographic parity produces similar interview-recommendation rates for qualified candidates from all demographic groups.
- **Equal opportunity:** True positive rates should be equal across groups. A loan approval model with equal opportunity correctly approves qualified applicants at the same rate regardless of demographic group.
- **Calibration:** Predicted probabilities should reflect actual outcome frequencies for all demographic groups. A model that predicts 80% approval probability should have approximately 80% of those cases actually approved, for every group.

Bias can be present even when no demographic features are included in the model's input features — proxy variables (zip codes, names, device types) can correlate strongly with protected characteristics. Testing must be performed on model outputs stratified by demographic group, not just by examining input features.

EU AI Act Requirement: High-risk AI systems — including those used in hiring, credit scoring, medical diagnosis, and law enforcement — must document fairness testing results. Model CI/CD pipelines should produce and retain these records automatically.

7. Performance and Latency Validation

A model that achieves excellent accuracy but cannot serve predictions within the latency requirements of the application it powers is not deployable. Performance validation is a mandatory gate in model CI/CD pipelines.

- Latency validation: Model inference must complete within the SLA defined by the application. If a fraud detection API must respond in under 100ms, a model that takes 300ms on test infrastructure fails the gate regardless of its accuracy.
- Throughput validation: The model must handle peak concurrent request volumes without latency degradation. Batch request testing simulates peak load conditions.
- Resource utilisation: CPU, memory, and GPU consumption must remain within operational budget. Large transformer models can have dramatically different resource profiles than smaller models with similar accuracy.
- Cold start validation: For serverless or container-based deployment, model loading time from cold state must meet requirements — some large models take minutes to load.

For transformer-based models in particular, there is a fundamental tension between model size (and accuracy) and inference speed. Model CI/CD pipelines quantify this trade-off through systematic testing rather than leaving it to be discovered in production.

8. Adversarial Robustness Testing

AI models are vulnerable to adversarial examples — carefully crafted inputs designed to cause misclassification. For security-critical models, adversarial robustness testing is not optional.

The threat is concrete and demonstrated:

- Computer vision: An image of a stop sign with specific pixel-level perturbations imperceptible to humans causes a vision model to classify it as a speed limit sign. Demonstrated on multiple production autonomous vehicle perception systems.
- Malware detection: A slight modification to a malware sample's binary structure — while preserving its malicious functionality — causes an AI-powered malware classifier to classify it as benign. Attackers systematically probe model boundaries to develop evasion techniques.
- Spam and phishing detection: Text adversarial attacks — substituting similar-looking Unicode characters, inserting invisible characters, or paraphrasing in specific ways — can cause spam detectors to pass malicious content.
- Fraud detection: Structuring transactions to appear similar to patterns the model was trained to classify as legitimate can evade fraud detection models.

Model CI/CD pipelines should include adversarial testing using frameworks like IBM's Adversarial Robustness Toolbox (ART) or CleverHans. Tests measure how much the model's accuracy degrades under defined adversarial attack scenarios, and gate deployment if degradation exceeds acceptable thresholds.

9. Model Drift Detection

Model drift — the degradation of model performance as real-world data distributions shift away from training distributions — is a normal failure mode unique to ML systems. Understanding both types of drift is exam-relevant.

Drift Type	Definition	Detection Method	Example
Feature drift (covariate shift)	Input feature distributions change	Statistical comparison of live inputs to training distribution (KS test, PSI)	Customer behaviour model after a major market event changes purchasing patterns
Label drift (concept drift)	Relationship between features and labels changes	Monitoring prediction confidence distributions; accuracy on labelled production samples	Phishing detector as attacker techniques evolve to match legitimate email patterns
Prediction drift	Model output distribution shifts without obvious cause	Statistical monitoring of output distributions over time	Fraud model score distribution shifting as fraud rates change seasonally

Drift detection requires continuous monitoring of both input feature distributions (compared to training data baselines) and model output distributions. Sudden drops in prediction confidence — the model becoming uncertain on inputs it previously handled confidently — often signal concept drift. Automated alerts trigger retraining pipelines or human review when drift metrics exceed defined thresholds.

10. Training Data Poisoning and Model Pipeline Attacks

The model CI/CD pipeline introduces attack vectors that do not exist in traditional application pipelines. Security teams must apply threat modelling discipline to model infrastructure.

- **Training data poisoning:** Injecting adversarial examples into training datasets to cause the model to learn biased or backdoored behaviour. A poisoned fraud detection model trained on tampered transaction data may learn to ignore fraud patterns from specific merchant categories. Mitigation requires data provenance controls, anomaly detection in training data, and data integrity verification.
- **Model registry compromise:** The model registry — the central store from which models are loaded for serving — is analogous to an application artifact registry. A compromised registry that serves

tampered model weights is equivalent to a supply chain attack on model serving infrastructure. Model artefact signing and integrity verification are required mitigations.

- Model extraction attacks: Adversaries query a production model with carefully designed inputs to reconstruct an approximate copy of the model. The extracted copy can be used to develop targeted adversarial attacks offline without triggering rate limiting on the production API. Mitigations include output confidence score clipping, rate limiting, and query pattern monitoring.

Real-World Context — Membership Inference Attack (2017): Researchers demonstrated that machine learning models trained on sensitive data (medical records, financial data) leak information about their training data through their confidence scores. Membership inference attacks can determine whether a specific individual's data was used to train the model — a significant privacy violation when the training dataset itself is sensitive. Production model CI/CD pipelines for sensitive domains should include differential privacy verification as part of the deployment gate.

11. A/B Testing and Canary Deployment for Models

Deploying a new model to 100% of production traffic without live validation is unnecessarily risky. A/B testing and canary deployment apply to model deployment with important adaptations for the probabilistic nature of model evaluation.

In a canary deployment, the new model serves a small percentage of live traffic (typically 5-10%) while the existing model continues to serve the remainder. The new model's performance is evaluated against real-world outcomes over a defined observation window. The key challenge is ground truth latency: for many model types, the correct answer is not immediately available. A credit decision made today may not be validated as correct or incorrect until the loan matures months later. Proxy metrics — downstream business indicators that correlate with model quality — must be used in the interim.

Automated rollback triggers revert to the previous model version if performance metrics degrade beyond defined thresholds during the canary phase. The gradual traffic expansion (5% -> 25% -> 50% -> 100%) limits exposure while providing the live production signal that synthetic test environments cannot fully replicate.

12. Explainability Validation

For many regulated applications, model predictions must be explainable — the decision rationale must be articulable in terms a human can understand and a regulator can audit. Explainability validation is an increasingly mandatory component of model CI/CD pipelines.

- Credit denial (FCRA, ECOA): Financial institutions in the US must provide adverse action notices explaining why credit was denied. AI models making credit decisions must produce explanation outputs that satisfy this requirement.
- Medical diagnosis support (EU AI Act): AI systems providing diagnostic support in clinical settings require audit trails of decision factors.
- Hiring decision support: AI-powered candidate screening systems face scrutiny from employment discrimination regulators in multiple jurisdictions.

Model CI/CD pipelines should validate that the model can produce explanations using SHAP (SHapley Additive exPlanations), LIME (Local Interpretable Model-agnostic Explanations), or built-in feature importance mechanisms; that explanations are stable and consistent across similar inputs; and that explanation outputs meet the specific regulatory format requirements of the applicable jurisdiction. A model that achieves excellent accuracy but cannot produce compliant explanations may be legally unusable in regulated contexts.

13. MLOps Platform Tools

The MLOps tooling ecosystem has matured into a set of production-grade platforms. For SecAI+ purposes, understand the category each tool belongs to rather than memorising feature details.

Tool	Category	Primary Capability
MLflow	Experiment tracking + model registry	Version control for models, parameters, and evaluation metrics
Kubeflow	ML pipeline orchestration	Kubernetes-native ML workflow automation and CI/CD integration
Amazon SageMaker	End-to-end MLOps (AWS)	Training, monitoring, A/B testing, and drift detection on AWS
Azure Machine Learning	End-to-end MLOps (Azure)	Integrated ML lifecycle management within Azure ecosystem
Evidently AI	Model monitoring and drift detection	Specialised production monitoring for data and model drift
Weights & Biases	Experiment tracking and visualisation	Model performance comparison and training visualisation

These platforms operationalise AI governance requirements by connecting policy commitments — models must be accurate, fair, and explainable — to automated enforcement at every deployment gate.

14. Versioning, Reproducibility, and Audit Trails

Reproducibility — the ability to regenerate an identical model from specified inputs — is both a debugging necessity and a compliance requirement in regulated industries. Achieving reproducibility requires tracking more artefacts than traditional software version control manages.

- **Model code version:** The specific model architecture code and training script, pinned to an exact commit hash.
- **Training data version:** The exact dataset used, including any preprocessing steps — typically managed with tools like DVC (Data Version Control) or Delta Lake.
- **Hyperparameter configuration:** Every parameter controlling training — learning rate, batch size, regularisation coefficients, architecture dimensions.
- **Environment specification:** Library versions (numpy, scikit-learn, PyTorch), Python version, hardware type (GPU model matters for reproducibility).
- **Random seeds:** All random number generator seeds used in data shuffling, weight initialisation, and dropout.

- Evaluation results: The metrics produced when the model was evaluated, so audit trails can compare historical performance.

Audit trails showing how each production model was produced — by whom, using what data, evaluated against what criteria, approved through what process — are increasingly required by AI governance regulations. MLflow and DVC provide the infrastructure for this comprehensive lineage tracking. Version control for models is not optional hygiene; it is the foundation of accountable, auditable AI deployment.

Key Terms Glossary

Term	Definition
MLOps	Machine Learning Operations — the application of DevOps principles (automation, versioning, CI/CD) to the ML model lifecycle.
Model drift	Degradation of model performance over time as real-world data distributions shift away from training distributions.
Feature drift (covariate shift)	Change in the statistical distribution of model input features compared to training data.
Concept drift (label drift)	Change in the relationship between input features and correct output labels — the model's learned mapping becomes incorrect.
Holdout dataset	Data reserved before training begins, used only for final evaluation — not for training or hyperparameter tuning.
Demographic parity	A fairness metric requiring equal positive prediction rates across demographic groups.
Equal opportunity	A fairness metric requiring equal true positive rates across demographic groups.
Adversarial example	A carefully crafted input designed to cause a model to produce an incorrect output, typically imperceptible to humans.
Training data poisoning	Injecting adversarial examples into training data to cause the model to learn incorrect or backdoored behaviour.
Model extraction attack	Querying a production model to reconstruct an approximate copy, enabling offline adversarial attack development.
SHAP	SHapley Additive exPlanations — a technique for explaining individual model predictions by attributing prediction to input features.
Canary deployment	Routing a small percentage of production traffic to a new model version to validate performance before full deployment.

Quick Revision Checklist

- ML models require different testing because behaviour emerges from training data (not just code), outputs are probabilistic, and models degrade over time without code changes.
- MLOps applies CI/CD discipline to the ML lifecycle: versioning, automated testing, gated deployment, and drift monitoring.
- Regulatory requirements (EU AI Act, NIST AI RMF) drive model testing from policy commitment to operational practice.

- Holdout datasets must be reserved before training — using training data for final evaluation inflates accuracy estimates (data leakage).
- Accuracy gates compare new models against both minimum absolute thresholds and current production model performance.
- Bias testing evaluates demographic parity, equal opportunity, and calibration across protected characteristic groups.
- Performance gates validate inference latency, throughput, and resource utilisation against SLA requirements.
- Adversarial robustness testing is mandatory for security-critical models — malware classifiers, fraud detectors, access control systems.
- Model drift detection monitors input feature distributions and prediction distributions continuously in production.
- Model pipeline attack surfaces include training data poisoning, model registry compromise, and model extraction — all require specific mitigations.
- Canary deployment routes a small traffic percentage to new models, with automated rollback if performance degrades.
- Reproducibility requires versioning code, training data, hyperparameters, environment, random seeds, and evaluation results together.

10 Exam-Based MCQs — Model Testing in CI/CD

These questions are written in real exam style — scenario-heavy, with plausible distractors. Read every explanation, including for questions you answered correctly.

Q1. Scenario: A payment processor's fraud detection model initially achieved 89% recall when deployed 14 months ago. The model code and serving infrastructure have not changed. A routine audit finds recall has degraded to 71% over this period while fraud loss rates have increased by 35%. A payment processor's fraud detection model initially achieved 89% recall. Fourteen months later, recall has degraded to 71% despite no changes to the model code. What is the MOST likely explanation?

- A) A developer introduced a bug in the model serving infrastructure
- B) The model experienced concept drift as fraud techniques evolved beyond the training data distribution
- C) The holdout evaluation dataset has become corrupted over time
- D) The model was retrained too frequently, causing overfitting to recent data

Answer: B **Explanation:** B is correct because the scenario describes gradual performance degradation over 14 months without code changes — this is concept drift, where the relationship between transaction features and the fraud label has changed as attackers adopted new techniques. A is wrong because code/infrastructure changes would cause sudden, not gradual, degradation. C is wrong because holdout dataset corruption would affect evaluation metrics, not live production recall. D is wrong because the scenario states the model was not retrained frequently.

Q2. Which data management practice is MOST critical to obtaining valid model accuracy measurements during CI/CD pipeline evaluation?

- A) Using the same data for training and evaluation to ensure the model is tested on familiar patterns
- B) Reserving a holdout dataset before training begins that is never used for training or hyperparameter tuning
- C) Evaluating the model on the most recent 20% of available data regardless of how the model was trained
- D) Using synthetic data generated by a GAN to ensure privacy in evaluation datasets

Answer: B **Explanation:** B is correct because a holdout dataset reserved before training begins ensures evaluation data was never seen during training or hyperparameter selection — producing an unbiased estimate of generalisation performance. A describes data leakage, the most common cause of inflated accuracy estimates. C uses temporal splitting but still risks leakage if this 20% influenced hyperparameter choices. D uses synthetic data which may not reflect real distribution properties accurately.

Q3. *Scenario:* A company deployed an AI hiring assistant to screen resumes for software engineering roles. HR notices that female candidates who meet all the listed qualifications receive interview recommendations at roughly 60% of the rate of equally qualified male candidates. A hiring algorithm produces interview recommendations at significantly lower rates for female candidates with equivalent qualifications to male candidates. Which fairness metric violation does this scenario MOST directly represent?

- A) Equal opportunity violation — true positive rates differ across gender groups
- B) Calibration failure — predicted probabilities do not reflect actual hire rates
- C) Precision failure — the model has high false positive rates for one group
- D) Coverage failure — the model was not trained on sufficient female candidate examples

Answer: A **Explanation:** A is correct because equal opportunity requires equal true positive rates across demographic groups. If qualified female candidates receive recommendations at lower rates than equally qualified male candidates, the true positive rate (correctly recommending qualified candidates) is lower for women — an equal opportunity violation. B (calibration) is about whether predicted scores match actual outcomes, not about group disparity in outcomes. C (precision) relates to false positive rates, not true positive rates. D is a possible cause, not a fairness metric name.

Q4. *Scenario:* A security researcher reports that a competitor has been sending large volumes of synthetic transactions to a payment processor's fraud API over several months, systematically probing the full range of its scoring behaviour. An attacker systematically queries a production fraud detection API with thousands of transactions to reconstruct an approximate copy of the deployed model. What type of attack is this?

- A) Training data poisoning
- B) Adversarial example injection
- C) Model extraction attack

D) Membership inference attack

Answer: C **Explanation:** C is correct because model extraction involves querying a production model extensively to reconstruct an approximate functional copy — enabling offline adversarial attack development without triggering the production system's rate limiting or anomaly detection. A (data poisoning) targets the training pipeline, not the serving API. B (adversarial example injection) crafts inputs to cause misclassification, not to copy the model. D (membership inference) determines whether specific data was in the training set, not to reconstruct the model.

Q5. A new fraud detection model achieves 96.5% precision compared to the production model's 97.1% precision but has 91.3% recall versus the production model's 89.0% recall. What should the CI/CD pipeline gate do?

- A) Automatically reject the new model because its precision is lower than the production model
- B) Automatically approve the new model because its recall is higher than the production model
- C) Flag the model for human review, presenting the precision/recall trade-off for an explicit deployment decision
- D) Retrain the model with higher regularisation to improve precision before re-evaluation

Answer: C **Explanation:** C is correct because the new model has a meaningful precision/recall trade-off compared to production — lower precision (more false positives) but higher recall (fewer missed fraud cases). Neither automatic approval nor automatic rejection is appropriate because the trade-off involves business judgment about the relative cost of false positives versus missed fraud. This is a human decision, not an automated one. A and B each consider only one metric. D assumes the precision gap requires a code fix rather than a deployment decision.

Q6. A regulated financial institution is deploying an AI credit scoring model. Which component of model CI/CD is MOST directly required by the Fair Credit Reporting Act (FCRA)?

- A) Adversarial robustness testing against credit application manipulation attacks
- B) Explainability validation ensuring the model can produce adverse action notices explaining rejection reasons
- C) Drift detection to identify when the model's training data has become outdated
- D) Canary deployment to limit exposure if the new model has unforeseen issues

Answer: B **Explanation:** B is correct because FCRA requires that applicants who are denied credit receive adverse action notices explaining the reasons for denial. An AI credit model must be able to generate these explanations — explainability validation confirms this capability in the CI/CD pipeline. A addresses security but is not directly mandated by FCRA. C addresses model quality but is not the FCRA-specific requirement. D is a deployment safety practice, not a FCRA requirement.

Q7. Which of the following BEST describes the difference between feature drift and concept drift?

- A) Feature drift affects classification models; concept drift affects regression models

- B) Feature drift is the change in input distributions; concept drift is the change in the relationship between inputs and labels
- C) Feature drift is detectable in CI/CD pipelines; concept drift can only be detected in production
- D) Feature drift is caused by data quality issues; concept drift is caused by model architecture problems

Answer: B **Explanation:** B is correct because feature drift (covariate shift) is when the distribution of input features changes relative to training — the model sees different types of inputs than it was trained on. Concept drift is when the relationship between features and the correct label changes — what was once a signal for fraud is no longer, or vice versa. A is wrong because both types affect all model types. C is wrong because both can be detected in production monitoring. D incorrectly attributes causes to infrastructure and architecture rather than data distribution changes.

Q8. A malware classification model achieves 99.2% accuracy on the holdout dataset. Security researchers demonstrate that applying specific binary transformations to known malware samples causes the model to classify them as benign, while the malware remains functional. What type of testing would MOST directly address this vulnerability?

- A) Bias and fairness testing to verify the model treats all malware families consistently
- B) Adversarial robustness testing using frameworks like IBM ART or CleverHans
- C) Canary deployment to monitor for performance degradation in production
- D) Drift detection to identify when the model's training distribution has shifted

Answer: B **Explanation:** B is correct because the attack described is an adversarial example — crafted inputs designed to cause misclassification while preserving malicious functionality. Adversarial robustness testing using frameworks like IBM's Adversarial Robustness Toolbox specifically measures model resistance to these attacks. A (bias testing) evaluates demographic fairness, not adversarial robustness. C (canary deployment) validates production performance gradually but does not test adversarial resilience. D (drift detection) monitors distribution change, not adversarial vulnerability.

Q9. An organisation wants to reproduce a model that was deployed in production 18 months ago for a regulatory audit. The model code is in version control, but the audit fails because the reproduced model produces different predictions. What is the MOST likely missing element?

- A) The model architecture was not documented clearly enough in the code comments
- B) The training data version, hyperparameter configuration, random seeds, or environment specifications were not captured alongside the model code
- C) The holdout evaluation dataset has been modified since the original evaluation
- D) The production model was compressed before deployment and the original weights were not retained

Answer: B **Explanation:** B is correct because ML reproducibility requires capturing not just the model code but also the exact training data version, hyperparameter configuration, random seeds, and compute environment (library versions, hardware). Without these, even identical code produces different models because training is stochastic. A is a documentation quality issue that doesn't affect reproducibility. C

affects evaluation validity but not model reproduction. D is possible but the question indicates code version control exists — the more common failure mode is missing the other artefacts listed in B.

Q10. A data scientist proposes deploying a new recommendation model directly to 100% of production traffic because it outperformed the current model on holdout data by 4%. What is the MOST significant risk this deployment approach fails to mitigate?

- A) The new model may not meet the organisation's bias and fairness requirements
- B) The new model may behave differently on live production data than on holdout data, with no ability to limit impact or revert quickly
- C) The new model's accuracy improvement may not justify the additional compute cost
- D) The new model may not be reproducible from version control without the training data

Answer: B **Explanation:** B is correct because the core risk of full immediate deployment is that holdout evaluation does not perfectly represent production conditions — edge cases, input distribution shifts, and interaction effects not present in the holdout can cause unexpected degradation. Canary deployment limits this exposure: only a small percentage of users experience the new model initially, performance is monitored against real outcomes, and rollback is fast. A (bias) is a valid concern but should have been caught pre-deployment. C (cost) is operational, not a deployment safety risk. D (reproducibility) is a governance concern, not a deployment risk.